Mentimeter

Haskell: Evaluare leneșă, programare point-free și list comprehensions

Mentimeter

List comprehension

- [expresie | value <- list, check value]
- un mod de a genera liste prin mentionarea domeniului valorilor + a conditiilor care trebuie verificate

Mentimeter

```haskell
-- numerele naturale pot fi generate cu [0 ..] sau [0, 1 ..]
even_naturals = [x | x <- [0 ..], even x]

bigger_than_x = [(x, y) | x <- [0 ..], y <- [x + 1 ..]]
-- sau bigger_than_x = [(x, y) | x <- [0 ..], y <- [0 ..], x < y]
```

Exemple



Mentimeter

```haskell
-- varianta 1) ok
-- ca sa facem fibonacci ne trebuie suma a doua liste
sum_lists l1 l2
    | null l1 = l2
    | null l2 = l1
    | otherwise = (head l1 + head l2) : sum_lists (tail l1) (tail l2)

-- primele 2 valori(0, 1) + restul sunt generate ca suma a 2 streamuri
fibonacci1 = 0 : 1 : sum_lists fibonacci1 (tail fibonacci1)
```

Exemplu mai fancy - ok

Mentimeter

```haskell
-- varianta 2) not ok
-- la list comprehension sunt **toate** combinatiile dintre a si b
-- ex: [(a, b) | a <- [0, 1, 2], b <- [3, 4, 5]]
--      = [(0, 3), (0, 4), (0, 5),
--         (1, 3), (1, 4), (1, 5),
--         (2, 3), (2, 4), (2, 5)]
-- daca incerc sa fac fibonacci asa atunci o sa obtin 0 si numai de 1
-- pentru ca pentru fiecare valoare a lui a caut toate valorile lui b
-- dintr-un sir infinit (**nu mai termin**)
--                  ex: a = 0 => b = 1 => fibonacci2 = [0, 1, 1]
-- (dupa b ia noua valoare 1) => b = 1 => fibonacci2 = [0, 1, 1, 1]
-- (si tot asa o sa am [0, 1, 1, ..., 1]
fibonacci2 = 0 : 1 : [a + b | a <- fibonacci2, b <- tail fibonacci2]
```

Exemplu mai fancy - not ok

Mentimeter

Liste infinite si generarea lor (++ functionale)

→ putem avea liste infinite pentru ca evaluare lenesa

Mentimeter

iterate :: (a -> a) -> a -> [a]

- iterate f x = [x, f x, f (f x), ...]
- ex: putem genera numerele naturale cu iterate (+ 1) 0

Mentimeter

repeat :: a -> [a]

- repeat x = [x, x, x, ...]
- intoarce o lista infinita cu aceeasi valoare

Mentimeter

intersperse :: a -> [a] -> [a]

- intersperse x [y1, y2, y3, ...] = [y1, x, y2, x, y3, x, ...]
- adauga un element intre elementele unei alte liste
- (trebuie si un import Data.List pentru a include functia)

Mentimeter

Alte metode (vezi lab 6)

zip, zipWith, foldl, foldr, map, filter

Mentimeter

```haskell
only_ones = iterate (\l -> 1:l) []
fibonacci = 0 : 1 : zipWith (+) fibonacci (tail fibonacci)
palindromes = filter isPalindrome [0 ..]
    where isPalindrome x = show x == reverse (show x)
```

Mentimeter

Point-free programming

- nu mentionam explicit parametrii
- scriem functiile ca succesiuni de compuneri de alte functii
- ne folosim de faptul ca functiile in haskell sunt curry by default

Mentimeter

(.) :: (b -> c) -> (a -> b) -> a -> c

- primeste 2 functii si intoarce compunerea lor
- ordinea se deduce din tipuri (prima e b->c, a doua a->b => ce scoate a doua functie e input la prima)
- (f . g) x = f (g x)

Mentimeter

($) :: (a -> b) -> a -> b

- aplica o functie unara (a->b) pe valoarea primita (de tip a)
- f $ x = f x, daar **impune gruparea expresiei: de la $ pana la sfarsitul expresiei curente este intr-o paranteza**

Mentimeter

flip :: (a -> b -> c) -> b -> a -> c

- flip schimba ordinea parametrilor primiti
- ex: daca salvam g = flip f atunci o sa apelam g x y si g y x

Mentimeter

```haskell
-- Avem f1 x = 2 * x + 1 pe care o putem rescrie ca (2 * x) + 1
-- * pe inputul x care e prima operatie facuta? * 2
-- * pe rezultat ce mai facem? + 1
-- Solutie: f1 x = (+ 1) . (* 2) x si putem scapa de x
f1 :: Int -> Int
f1 = (+ 1) . (* 2)

-- Avem f2 n = take (f1 n) fibonacci care scoate primele f1 n
-- elemente din fibonacci.
-- * daca vrem sa o scriem point free atunci ar trebui sa scapam de n
--   mutandu-l mai la dreapta
-- * putem sa facem flip pe take: f2 n = flip take fibonacci (f1 n)
-- * putem scapa si de paranteza acum: f2 n = flip take fibonacci $ f1 n
-- * si acum putem scapa si de n: f2 = flip take fibonacci . f1
--   (nu e ok $ pentru ca e echivalent cu flip take fibonacci (f1))
f2 :: Int -> [Int]
f2 = flip take fibonacci . f1
```

Exemple 0

Mentimeter

```haskell
-- multe moduri de a aplica ./$
list = [1, -9, 100, 23, 5, 12]
sum_of_square_of_odds l = sum (map (\x -> x * x) (filter odd l))

s1 l = sum $ map (\x -> x * x) $ filter odd l
s2 l = (sum . map (\x -> x * x) . filter odd) l

s3 = sum . map (\x -> x * x) . filter odd
```

Exemple 1